FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
JESÚS OVIEDO PÉREZ

# INFORME TÉCNICO BREVE

Nicolás Andrés Chica Córdoba

Juan Diego Bonilla Aroca

Tutor

Juan José Ramos Diaz

Fundación Escuela Tecnológica de Neiva

Ingeniería de software

Rivera-Huila

2026

## 1. Introducción

ArquiSelect es un sistema experto que recomienda un estilo de arquitectura de software a partir de los requisitos del proyecto. El conocimiento vive en SWI-Prolog (hechos, reglas y un analizador DCG) y Python solo aporta la interfaz y el puente PySwip. Este informe consolida tres cosas: la arquitectura final del software, el árbol lógico principal de la inferencia y el reporte de las pruebas de estrés aplicadas al manejo de contradicciones.

## 2. Arquitectura final del software

El sistema se divide en dos capas con una frontera estricta: Python no decide nada, solo envía datos y muestra lo que Prolog responde. La Figura 1 resume los componentes y sus conexiones.

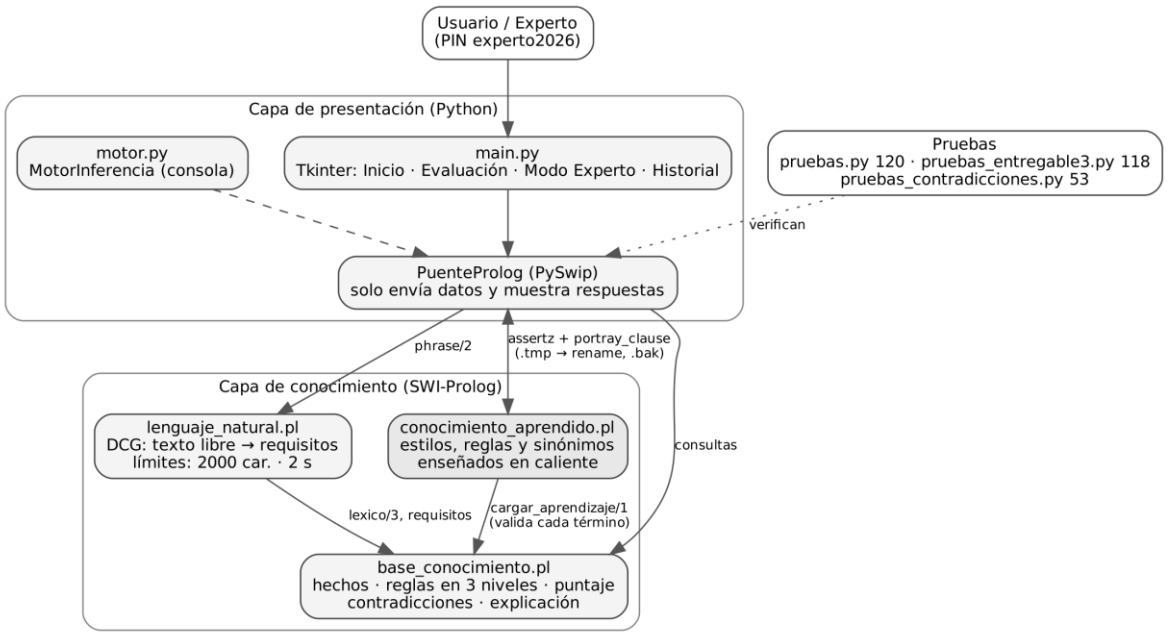



*Figura 1. Arquitectura final de ArquiSelect.*

| Componente | Responsabilidad |
|---|---|
| `base_conocimiento.pl` | Hechos de dominio, reglas en tres niveles (requisito → atributo → estilo), puntaje, prioridad, riesgo, explicación ("¿Por qué?") y módulo de contradicciones. |
| `lenguaje_natural.pl` | Gramática DCG que convierte texto libre en requisitos y negados; léxico dinámico (`lexico/3`); límites de 2000 caracteres y 2 s. |
| `conocimiento_aprendido.pl` | Estilos, reglas expertas y sinónimos enseñados en caliente por el experto; se escribe de forma atómica (`.tmp` + renombrado, copia `.bak`) y se recarga validando cada término. |
| `main.py` | Interfaz Tkinter: Inicio con rol Usuario/Experto (PIN con hash sha256), Evaluación, Modo Experto (4 pestañas) e Historial; clase `PuenteProlog`. |
| `motor.py` | Ruta de consola: `MotorInferencia` consulta el mismo `diagnosticar_arquitectura/9`. |

**Tamaño de la base de conocimiento** (conteo sobre los archivos entregados):

| Archivo | Hechos | Reglas | Total cláusulas |
|---|---|---|---|
| `base_conocimiento.pl` | 306 | 250 | 556 |
| `lenguaje_natural.pl` | 232 | 141 | 373 |
| **Total** | **538** | **391** | **929** |

Dentro de la base: 20 requisitos (`requisito_categoria/2`), 11 estilos (`estilo/2`), 8 incompatibilidades (`incompatible/2`), 19 vínculos requisito–atributo, 64 aportes (`aporta/3`), 28 penalizaciones (`penaliza/3`) y 162 entradas de léxico (`lexico/3`).

## 3. Árbol lógico principal

El predicado de entrada es `diagnosticar_arquitectura/9`, con dos cláusulas que Prolog prueba en orden. La primera es el bloqueo por contradicción dura: si existe, fija `Mejor = ninguno` y un corte (`!`) impide llegar a la segunda. La segunda es el diagnóstico normal. La Figura 2 muestra el árbol curado a partir del árbol real generado desde el código.

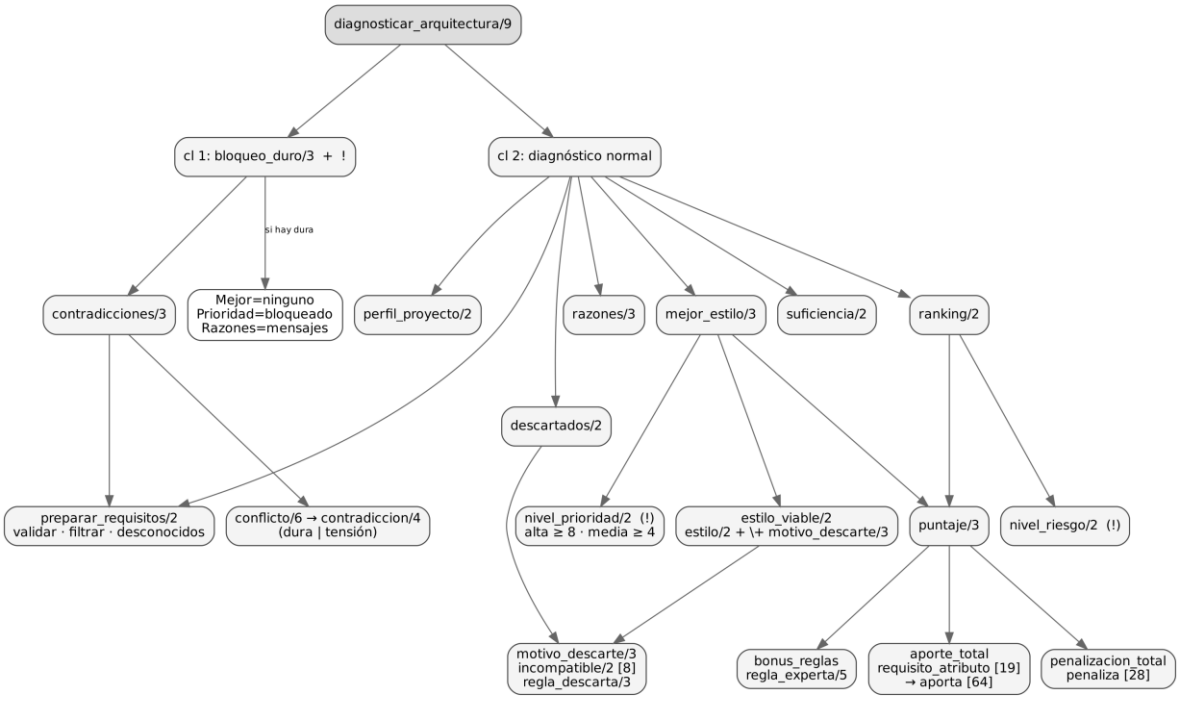



*Figura 2. Árbol lógico principal de la inferencia.*

**Cómo razona.** Los requisitos se normalizan en `preparar_requisitos/2` (minúsculas, sin repetidos, desconocidos aparte). Luego `puntaje/3` suma el aporte de cada requisito a los atributos de calidad (`aporte_total`), resta las penalizaciones y añade el bonus de las reglas expertas: `puntaje = aporte_total - penalizacion_total + bonus_reglas`. `estilo_viable/2` usa negación como fallo (`\+ motivo_descarte/3`) para eliminar estilos incompatibles. Los cortes en `nivel_prioridad/2` (alta ≥ 8, media ≥ 4) y `nivel_riesgo/2` hacen excluyentes sus cláusulas.

**Estados del dictamen:** vacío, insuficiente, empate (umbral de desempate 2), definitivo y contradictorio. El último tiene confianza nula y no emite recomendación.

## 4. Pruebas de estrés: manejo de contradicciones

### 4.1 Diseño

Los pares de requisitos incompatibles se declaran como hechos `contradiccion(A, B, Tipo, Mensaje)`. Una contradicción **dura** (por ejemplo `equipo_pequeno` con `equipo_grande`, o `latencia_critica` con `procesos_largos`) bloquea el diagnóstico; una **tensión** (por ejemplo `disponibilidad_alta` con `presupuesto_bajo`) solo advierte y el sistema sigue recomendando. `consistencia_entrada/4` devuelve `consistente`, `advertencia` o `bloqueado`.

### 4.2 Resultados

Se re-ejecutaron los casos contra la base entregada con SWI-Prolog 9.0.4 y PySwip. En paralelo se conserva la evidencia de la suite `pruebas_contradicciones.py` (53/53 verificaciones, Windows 11, SWI-Prolog 10.0.2).

| Grupo | Qué se estresa | Resultado |
|---|---|---|
| A1 | Los 3 pares duros, en ambos órdenes y mezclados con otros requisitos (6 casos) | `Mejor = ninguno`, `Prioridad = bloqueado`, ranking vacío, estado `contradictorio` |
| A2 | Requisito afirmado y negado a la vez | `consistencia_entrada` → `bloqueado`; ver limitación en 4.3 |
| A4 | Cláusula “steadfast”: llamar con `Mejor` ya fijado | Falla; no se salta el bloqueo |
| B1 | Las 2 tensiones, en ambos órdenes (4 casos) | `advertencia` y sí hay recomendación (p. ej. `eventos`, `cliente_servidor`) |
| C1–C4 | Caso normal, mayúsculas, repetidos, dura + tensión juntas, entradas que no son lista | `consistente`; una sola contradicción por par repetido; manda la dura y se listan ambas; sin excepción |
| D1 | Texto libre → mismo bloqueo o aviso (“un equipo pequeño y un equipo grande”) | Requisitos extraídos por el DCG y estado `bloqueado` / `advertencia` correctos |

### 4.3 Entradas hostiles y límites

| Entrada | Resultado |
|---|---|
| Vacío, solo espacios/tab/salto, solo emojis | Sin requisitos, sin excepción, Prolog sigue activo |
| Intento de inyección (`x), halt. %`, comillas, barra + comilla) | Tratado como texto: sin requisitos, Prolog sigue activo |
| Emoji o caracteres de control en medio de una frase válida | Los requisitos válidos se extraen igual |
| Texto de 4500 caracteres | Rechazado con aviso, sin análisis |
| 2000 vs 2001 caracteres | 2000 se analiza; 2001 devuelve "texto demasiado largo: máximo 2000 caracteres" |
| Límite de tiempo forzado a $1\times10^{-6}$ s | Análisis cortado con aviso "tiempo de análisis agotado"; al restaurar el límite vuelve a funcionar |

### 4.4 Hallazgos

- **Limitación documentada:** `diagnosticar_arquitectura/9` no recibe la lista de negados, por eso no ve un requisito "afirmado y negado a la vez". Lo detectan `consistencia_entrada/4` y `que_falta/6`, y la interfaz decide con el estado de este último, de modo que el usuario igual ve el bloqueo.
- La tensión nunca altera el ranking: solo informa. Esto evita bloquear proyectos reales que combinan requisitos difíciles pero posibles.
- Un par excluyente nuevo en `excluyentes/2` se vuelve contradicción dura sin tocar el módulo de contradicciones.

## 5. Resumen de pruebas y conclusiones

| Suite | Verificaciones | Resultado |
|---|---|---|
| `pruebas.py` (base y puente) | 120 | 120/120 |
| `pruebas_entregable3.py` (DCG, Modo Experto, persistencia) | 118 | 118/118 |
| `pruebas_contradicciones.py` (contradicciones y robustez) | 53 | 53/53 |
| **Total** | **291** | **291/291** |

La arquitectura mantiene el conocimiento en Prolog y la presentación en Python, el árbol lógico queda trazable hasta los hechos que lo sostienen, y el manejo de contradicciones bloquea o advierte según la gravedad sin romperse ante entradas hostiles. Las cifras de las suites 120, 118 y 53 provienen de las ejecuciones del equipo; los conteos de cláusulas y los casos de la sección 4 se verificaron de nuevo para este informe.